MODULAR CALCULATOR

SHORT REFLECTION

1. What functions did you create in your calculator program?

- I created four functions: `add_numbers()`, `subtract_numbers()`, `multiply_numbers()`, and `divide_numbers()`. Each function performs a specific arithmetic operation, allowing the calculator to execute calculations in an organized manner.

2. What parameters did your functions use?

- Each function used two parameters, `num1` and `num2`, which represent the two numbers needed to perform the selected arithmetic operation.

3. What arguments were passed when the functions were called?

- The arguments passed to the functions were the two numbers entered by the user, stored in the variables `num1` and `num2`. These arguments provide the actual values that the functions use to perform calculations.

4. How did your program use the returned value?

- The program stored the value returned by the selected function in the variable `result`. It then displayed the calculated answer to the user. In the division function, the program also checked whether the returned value was `None`, indicating that division by zero had occurred, before displaying the appropriate error message.

5. Why is it better to divide the calculator program into functions instead of writing everything in one long block of code?

- Dividing the program into functions makes the code more organized, readable, and easier to maintain. Each function handles a specific operation, making it easier to identify errors, test individual operations, and modify the program without affecting unrelated sections of the code. This modular approach also reduces code repetition and promotes code reusability.

By: *Iniego Miguel H. Sur*
From: *8 - Rosal*